

UNIVERSIDAD NACIONAL DEL ALTIPLANO

Puno

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas
Escuela Profesional de Ingeniería de Sistemas

Complejidad y búsquedas

Comparación teórica y experimental
Búsqueda lineal y binaria en Python y C++20

Curso: Algoritmos y Estructuras de Datos – SIS210
Semana: 2
Docente: ZANABRIA GALVEZ ALDO HERNAN

Integrantes

YANA MENDOZA CARLOS BENEDICTO
BELIZARIO YANA DAVID VICTOR
CURASI ZEVALLOS HANDDY RONALD

Repositorio GitHub:

github.com/handdycurasi-crypto/SIS210-Semana02-Complejidad

1. Problema y objetivo

Una colección de transacciones necesita localizar `CustomerID`. Se compara recorriéndola directamente con buscar en una copia previamente ordenada. El objetivo es contrastar el crecimiento teórico con sondeos y tiempos reales, incluyendo el costo de preparar el orden. Se sigue la guía de Zanabria Gálvez (2026). Aunque su portada dice C++17 y Semana 1, las instrucciones operativas corresponden a C++20 y Semana 2; se adoptan estas últimas.

2. Marco teórico profundizado

2.1. Tamaño de entrada n

El análisis de un algoritmo comienza definiendo qué magnitud representa el tamaño de la entrada, porque la función de costo solo tiene sentido cuando n está asociado de forma consistente con aquello que crece en el problema. En esta práctica, n representa el número de registros de `CustomerID` almacenados en la muestra, incluidos identificadores repetidos; no representa la cantidad de clientes distintos. Esta elección permite comparar la búsqueda lineal y la búsqueda binaria sobre colecciones del mismo tamaño y evita interpretar de forma ambigua una expresión como $T(n)$. Cormen et al. presentan el tamaño de entrada como la variable básica respecto de la cual se expresa el tiempo de ejecución, mientras Levitin y Skiena resaltan que su definición depende del problema concreto y de la representación elegida (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

La unidad utilizada para n también determina qué conclusiones son válidas. Por ejemplo, si se midiera número de clientes únicos en lugar de registros, las muestras con identificadores repetidos tendrían un tamaño diferente aun cuando el algoritmo recorriera exactamente la misma cantidad de posiciones de memoria. Por ello, en análisis de algoritmos conviene separar la dimensión lógica del problema de otras características de los datos, como distribución, duplicados o valor numérico de las claves. Esta disciplina permite comparar implementaciones sin modificar silenciosamente la variable independiente del experimento (Cormen et al., 2022; Sedgewick & Wayne, 2011; McGeoch, 2012).

2.2. Modelo RAM

El modelo RAM (Random Access Machine) abstrae una computadora ideal en la que operaciones elementales como acceso a memoria, asignación, comparación y aritmética básica se contabilizan con costo constante. Su utilidad no consiste en reproducir exactamente lo que ocurre en un procesador moderno, sino en proporcionar un lenguaje común para razonar sobre la cantidad de trabajo que realiza un algoritmo al crecer la entrada. Gracias a esta abstracción se puede comparar la estructura del costo de dos soluciones sin depender de la frecuencia de CPU, el compilador o el sistema operativo de una máquina particular (Cormen et al., 2022; Levitin, 2012; Knuth, 1997).

El modelo RAM exige declarar un convenio de conteo. Una misma línea de código puede descomponerse en accesos, comparaciones, sumas y asignaciones, y dos análisis pueden producir constantes diferentes sin contradecirse si ambos mantienen un criterio coherente. Lo relevante para el análisis asintótico es que las diferencias de costo constante no alteran el orden de crecimiento dominante. En esta práctica se conserva además un contador de sondeos separado del tiempo real, de manera que la evidencia estructural del algoritmo no se confunda con la velocidad de una plataforma específica (Cormen et al., 2022; Skiena, 2020; Zanabria Gálvez, 2026).

2.3. Función de costo $T(n)$

La función $T(n)$ representa el número de operaciones elementales ejecutadas por un algoritmo bajo un modelo de costo previamente definido. No es, por sí misma, una medición en segundos: es una expresión matemática que permite observar cómo cambia el trabajo al aumentar n . Derivar $T(n)$ obliga a identificar qué instrucciones se ejecutan una vez, cuáles dependen del número de iteraciones y cuáles solo aparecen bajo determinadas condiciones. Esta derivación es un puente entre el código concreto y las cotas asintóticas que luego resumen su comportamiento (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

Para búsqueda lineal, por ejemplo, la cantidad de posiciones inspeccionadas puede variar desde una hasta n . En un peor caso ausente, el ciclo debe visitar cada posición; si se adopta un convenio RAM que contabiliza inicializaciones, control del ciclo, acceso, comparación, actualización de contadores e incremento del índice, se obtiene una expresión de la forma $an + b$. El valor concreto de a y b depende del convenio, pero la dependencia lineal con n se mantiene. En búsqueda binaria, en cambio, la cantidad de intervalos posibles se reduce aproximadamente a la mitad en cada iteración, lo que conduce a una función relacionada con $\log_2 n$ (Cormen et al., 2022; Sedgewick & Wayne, 2011; Levitin, 2012).

2.4. Operación dominante y tasas de crecimiento

Cuando n aumenta, algunos términos de $T(n)$ influyen mucho más que otros. Si $T(n) = 7n + 5$, el término $7n$ domina el crecimiento y la constante aditiva deja de ser significativa para entradas grandes. De manera similar, en una expresión como $3n^2 + 4n + 20$, el término cuadrático domina a los lineales y constantes. Esta simplificación no afirma que las constantes sean irrelevantes en tiempos reales; afirma que, desde la perspectiva asintótica, el

tipo de crecimiento está determinado por el término que aumenta más rápidamente (Cormen et al., 2022; Levitin, 2012; Knuth, 1997).

Las tasas habituales 1 , $\log n$, n , $n \log n$ y n^2 permiten ordenar algoritmos según su escalabilidad teórica. Para tamaños pequeños, una implementación con peor orden asintótico puede ser más rápida por constantes favorables, menor sobrecarga o mejor localidad de memoria. Sin embargo, al crecer suficientemente la entrada, las diferencias entre tasas de crecimiento terminan dominando. Por eso el análisis asintótico se complementa, pero no se reemplaza, con mediciones empíricas (Cormen et al., 2022; Skiena, 2020; McGeoch, 2012).

2.5. Notación O

La notación $O(g(n))$ expresa una cota superior asintótica. Formalmente, $T(n) \in O(g(n))$ si existen constantes positivas c y n_0 tales que $0 \leq T(n) \leq c g(n)$ para todo $n \geq n_0$. La notación permite afirmar que, a partir de un tamaño suficientemente grande, el crecimiento de $T(n)$ no supera un múltiplo constante de $g(n)$. Por ejemplo, $7n + 5 \in O(n)$ porque puede acotarse por un múltiplo de n para todo n suficientemente grande (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

Un error frecuente consiste en usar O como sinónimo automático de “peor caso”. O describe una relación entre funciones y puede utilizarse para acotar una función de mejor caso, promedio o peor caso, siempre que se especifique cuál se está analizando. Por ejemplo, el mejor caso de una búsqueda lineal que encuentra la clave en la primera posición tiene costo constante y pertenece a $O(1)$, mientras que su peor caso pertenece a $O(n)$. La distinción entre cota y escenario es central en la guía de esta práctica (Cormen et al., 2022; Black, 2020; Zanabria Gálvez, 2026).

2.6. Notación Ω

La notación $\Omega(g(n))$ expresa una cota inferior asintótica. Se cumple cuando existen constantes positivas c y n_0 tales que $T(n) \geq c g(n)$ para todo $n \geq n_0$. Mientras O establece un techo asintótico, Ω establece un piso para la función analizada. Esta herramienta es útil para demostrar que una cantidad de trabajo no puede crecer por debajo de cierto orden dentro del modelo adoptado (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

También aquí es incorrecto identificar Ω con “mejor caso” de manera automática. El peor caso de búsqueda lineal, cuando la clave está ausente, no solo es $O(n)$; también es $\Omega(n)$ porque debe inspeccionar proporcionalmente n posiciones. Por tanto, ese mismo peor caso es $\Theta(n)$. La notación inferior describe la función de costo elegida, no el tipo de entrada que produjo dicha función (Cormen et al., 2022; Black, 2020; Zanabria Gálvez, 2026).

2.7. Notación Θ y cota ajustada

La notación $\Theta(g(n))$ expresa una cota ajustada: la función está acotada superior e inferiormente por múltiplos positivos de $g(n)$ a partir de cierto n_0 . En términos formales, existen $c_1, c_2 > 0$ y n_0 tales que $c_1 g(n) \leq T(n) \leq c_2 g(n)$ para todo $n \geq n_0$. Esto permite describir el orden de crecimiento con mayor precisión que una cota superior aislada (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

Decir que el peor caso de búsqueda binaria es $\Theta(\log n)$ significa que su número de iteraciones crece del mismo orden que un logaritmo, tanto por arriba como por abajo, bajo el modelo y escenario definidos. No significa que Θ sea sinónimo de “caso promedio”. Un promedio puede ser $\Theta(n)$, $\Theta(\log n)$ o cualquier otro orden dependiendo del algoritmo y de la distribución de entradas. Separar estas dos ideas evita una de las confusiones conceptuales más comunes al comenzar el análisis asintótico (Cormen et al., 2022; Sedgewick & Wayne, 2011; Zanabria Gálvez, 2026).

2.8. Mejor, promedio y peor caso

Los casos describen conjuntos o distribuciones de entradas, no tipos de notación. En búsqueda lineal, una clave ubicada en la primera posición produce el mejor escenario porque se necesita un solo sondeo; una clave ubicada al final o ausente produce el peor escenario porque se inspeccionan n posiciones. El caso promedio requiere declarar supuestos probabilísticos, por ejemplo que todas las posiciones de éxito sean equiprobables, en cuyo caso el número esperado de sondeos es $(n + 1)/2$ (Cormen et al., 2022; Levitin, 2012; Skiena, 2020).

En experimentación es importante no confundir la repetición de una clave con un promedio teórico. Ejecutar 30 veces la misma consulta permite observar variabilidad temporal bajo condiciones controladas, pero no transforma esa consulta en una muestra de todas las posiciones posibles. Esta práctica separa explícitamente los escenarios inicio, centro, final y ausente, de modo que las mediciones representan esos casos condicionados y no una distribución real de consultas de clientes (McGeoch, 2012; Cormen et al., 2022; Zanabria Gálvez, 2026).

2.9. Búsqueda lineal

La búsqueda lineal recorre secuencialmente una colección hasta hallar la clave o agotar los elementos. Su principal ventaja es que no requiere que los datos estén ordenados ni una preparación previa. En un arreglo, el procedimiento es directo y aprovecha un patrón de acceso secuencial. Su mejor caso requiere un sondeo, mientras que el peor caso

requiere n sondeos; bajo supuestos uniformes de éxito, su costo esperado sigue siendo lineal. La implementación iterativa utiliza $\Theta(1)$ memoria auxiliar (Cormen et al., 2022; Skiena, 2020; Black, 2020).

En esta práctica se modela un recorrido equivalente al código: inicializar n , índice $i = 0$ y contador $ops = 0$; mientras $i < n$, examinar $a[i]$, incrementar ops , comparar con la clave y avanzar si no coincide. Cada operación cuesta 1; incrementar una variable cuesta 2 (suma y asignación). Se consideran obtener/asignar la longitud almacenada y retornar el par como operaciones abstractas unitarias. Este convenio hace trazable la función de costo y, aunque otro convenio produciría constantes diferentes, el orden lineal no cambia (Cormen et al., 2022; Levitin, 2012; Zanabria Gálvez, 2026).

Operación	Ausente	Éxito en visita k
Inicializaciones de n , i y ops	3	3
Control $i < n$	$n + 1$	k
Acceso al elemento	n	k
Incremento de ops : suma y asignación	$2n$	$2k$
Comparación de igualdad	n	k
Incremento de i : suma y asignación	$2n$	$2(k - 1)$
Retorno	1	1
Total	$7n + 5$	$7k + 2$

Así, $T_{\text{ausente}}(n) = 7n + 5$ y, para $n \geq 1$, $7n \leq 7n + 5 \leq 12n$, por lo que el peor caso es $\Theta(n)$. El contador almacenado en los CSV mide únicamente sondeos, no todas las operaciones del convenio RAM: vale n en ausencia y k cuando el éxito ocurre en la visita k . Esta separación permite comparar la predicción estructural con el tiempo real sin presentar ambas métricas como si fueran equivalentes (Cormen et al., 2022; McGeoch, 2012; Zanabria Gálvez, 2026).

2.10. Búsqueda binaria

La búsqueda binaria requiere una colección ordenada y compara la clave con el elemento central del intervalo activo. Si no coincide, descarta la mitad que no puede contener la clave. La secuencia aproximada de tamaños es $n, n/2, n/4, n/8, \dots$; después de k divisiones queda $n/2^k$. Al imponer $n/2^k \leq 1$ se obtiene $k \geq \log_2 n$, lo que explica el crecimiento logarítmico del número de iteraciones (Cormen et al., 2022; Sedgewick & Wayne, 2011; Black, 2022).

Su mejor caso es $\Theta(1)$ cuando la primera posición central contiene la clave, mientras que el peor caso es $\Theta(\log n)$. Con un arreglo, la implementación iterativa utiliza $\Theta(1)$ memoria auxiliar. La ventaja asintótica frente a búsqueda lineal puede ser enorme para entradas grandes, pero depende de que los datos ya estén ordenados o de que el costo de ordenarlos pueda justificarse. Además, el acceso no secuencial y los costos concretos de la implementación influyen en el tiempo observado sin modificar la complejidad teórica (Cormen et al., 2022; Skiena, 2020; Levitin, 2012).

2.11. Costo de preparación y amortización

Comparar una búsqueda lineal sobre datos desordenados con una búsqueda binaria sobre datos ordenados exige contabilizar el costo de preparación. Si una colección debe ordenarse antes de la primera búsqueda, el costo total puede modelarse como $C_B(q) = S + qB$, donde S es el costo de ordenar, B el costo de una búsqueda binaria y q la cantidad de consultas. Para lineal, si no existe preparación equivalente, puede modelarse $C_L(q) = qL$. La comparación relevante no es solamente L frente a B , sino el costo acumulado de la estrategia completa (Cormen et al., 2022; Skiena, 2020; McGeoch, 2012).

Cuando $L > B$, la estrategia con ordenamiento empieza a ser favorable si $q > S/(L - B)$. Este umbral depende de los tiempos observados, del tipo de consulta y de la estabilidad de la colección; si los datos cambian continuamente, el costo de mantener el orden puede alterar la decisión. Por eso la búsqueda binaria no debe presentarse como universalmente superior: para una única consulta sobre datos desordenados, recorrer directamente puede ser más razonable; para miles de consultas sobre una colección estable, la preparación puede amortizarse (Levitin, 2012; Skiena, 2020; McGeoch, 2012).

2.12. Complejidad teórica, rendimiento empírico y reproducibilidad

La complejidad asintótica describe cómo escala una función de costo; el tiempo real incorpora además constantes, arquitectura, compilador, intérprete, representación de datos, caché, predicción de saltos, sistema operativo y carga concurrente. Por ello, un resultado experimental no “demuestra” una cota matemática. Lo correcto es verificar si las tendencias observadas son consistentes con la predicción teórica y analizar las discrepancias. Esta complementariedad entre análisis formal y experimentación es central en la evaluación rigurosa de algoritmos

(McGeoch, 2012; Cormen et al., 2022; Skiena, 2020).

La reproducibilidad exige documentar datos, versiones, semillas, región cronometrada, número de repeticiones y estadísticos. En Python, `perf_counter_ns()` proporciona un contador de alta resolución apropiado para medir intervalos; en C++, `steady_clock` es monotónico y está diseñado para medición de duraciones. Aun así, expresar un resultado en nanosegundos no implica precisión física de un nanosegundo, y las mediciones muy pequeñas pueden estar dominadas por sobrecarga del reloj o ruido. Por eso esta práctica conserva datos crudos, reporta mediana además de promedio y separa carga, ordenamiento y búsqueda (Python Software Foundation, s. f.; ISO/IEC JTC1/SC22/WG21, s. f.; McGeoch, 2012).

En conjunto, estos doce núcleos muestran que elegir un algoritmo no consiste únicamente en memorizar una notación. Es necesario definir la entrada, modelar operaciones, distinguir cotas de escenarios, considerar requisitos de los datos y comprobar empíricamente cómo se comporta una implementación real. La búsqueda lineal y la binaria son un caso de estudio especialmente útil porque presentan crecimientos teóricos diferentes y permiten observar cómo la preparación, el número de consultas y el entorno modifican la decisión práctica sin alterar las cotas matemáticas (Cormen et al., 2022; Levitin, 2012; McGeoch, 2012).

3. Dataset y preparación

Se descargó *Online Retail* (Chen, 2015), DOI [10.24432/C5BW33](https://doi.org/10.24432/C5BW33), de UCI, bajo licencia CC BY 4.0. Se usó `CustomerID`, convertido a entero. El archivo observado contiene 541 909 filas; se descartaron 135 080 identificadores faltantes y quedaron 406 829 registros válidos, con 4 372 clientes distintos. Aunque la ficha web indica ausencia de faltantes, estos conteos proceden del Excel real. No se filtraron anulaciones, países ni importes: el problema es localizar claves, no estimar ventas.

Los 402 457 registros adicionales respecto a clientes únicos se conservan; no equivalen necesariamente a filas completas duplicadas. Para cada n , la semilla es $21002 + n$. Hasta 100 000 se muestrean filas sin reemplazo. Para 500 000 se remuestrean filas válidas con reemplazo: no se presentan como transacciones independientes adicionales.

n	Semilla	Clientes distintos	Muestreo de filas
100	21 102	87	Sin reemplazo
1 000	22 002	703	Sin reemplazo
10 000	31 002	2 667	Sin reemplazo
100 000	121 002	4 182	Sin reemplazo
500 000	521 002	4 344	Con reemplazo

Escenarios exactos. En cada muestra se seleccionan tres claves que aparecen una sola vez y se intercambian con las posiciones 0, $n/2$ y $n - 1$. Se conserva el multiconjunto y se garantiza primera coincidencia al inicio, centro y final. La clave ausente es máximo de la muestra más uno y no se agrega a los datos. Este condicionamiento permite contrastar casos; no representa la distribución de consultas de clientes reales.

Los CSV conservan `fila_excel` y `CustomerID`. `metadatos.json` guarda limpieza, semillas y SHA-256 del origen y las muestras. Ambos lenguajes leen las mismas muestras y `claves.csv`. Lineal usa el orden preparado; binaria usa su copia ordenada. Los nombres de escenarios se refieren al orden original: el centro original no tiene por qué ser el centro de la copia ordenada.

4. Diseño experimental

La matriz tiene 5 tamaños $\times$ 2 algoritmos $\times$ 4 escenarios $\times$ 30 repeticiones $\times$ 2 lenguajes: **2 400 búsquedas individuales**. Hay un calentamiento por combinación y después se mezcla el orden de tareas con semilla $902 + n$ en cada lenguaje. Las implementaciones se ejecutaron consecutivamente, no simultáneamente. Se conserva cada repetición y se calcula promedio, mediana, mínimo y máximo de tiempo y sondeos.

Python usa `perf_counter_ns()` y C++ `steady_clock` con conversión a nanosegundos, conforme a su documentación (Python Software Foundation, s. f.; ISO/IEC JTC1/SC22/WG21, s. f.). Se cronometra la llamada instrumentada de búsqueda; carga, selección de función, copia, validación, ordenamiento e impresión quedan fuera. Ordenar se mide aparte, 30 veces por tamaño y lenguaje sobre una nueva copia: **300 ordenamientos**. La copia tampoco forma parte de ese tiempo.

5. Implementaciones y control de variables

Las búsquedas son manuales. Lineal retorna la primera coincidencia; binaria, una coincidencia de la copia ordenada. **comparaciones** conserva el convenio de la guía: un sondeo por elemento visitado. En binaria un sondeo puede ejecutar igualdad y menor-que; no es un conteo de todas las expresiones booleanas. Los resultados se verifican y se escriben fuera del intervalo; su uso observable evita tratar la búsqueda como cálculo sin efecto.

Se contrastaron 2 070 consultas por lenguaje con oráculos de biblioteca, incluyendo vacíos, repetidos, negativos y límites. Se probó rechazo de desorden y errores de preparación. El análisis cruza claves, posiciones y sondeos entre lenguajes. La demo pequeña está separada del benchmark y permite ingresar una clave, elegir algoritmo, provocar desorden, corregir y repetir.

Entorno real: Work remoto. Linux 6.18.35 x86_64; CPU visible AMD EPYC 9V74; memoria visible 16 399 700 kB, límite de contenedor 14 GiB; 9 CPU lógicas visibles y cuota equivalente a 8. Python 3.12.14, GCC 13.3.0; flags `-std=c++20 -O2 -Wall -Wextra -pedantic`. pandas 2.2.3, matplotlib 3.10.8 y openpyxl 3.1.5. No se atribuyen estas mediciones a la PC del estudiante.

6. Resultados teóricos y empíricos

6.1. Número de sondeos

Los 80 grupos tienen 30 repeticiones. Python y C++ coinciden en posición y sondeos para cada consulta. La figura representa Python; la correspondiente de C++ está incluida y tiene los mismos conteos. Los ejes logarítmicos permiten visualizar desde un sondeo hasta 500 000.

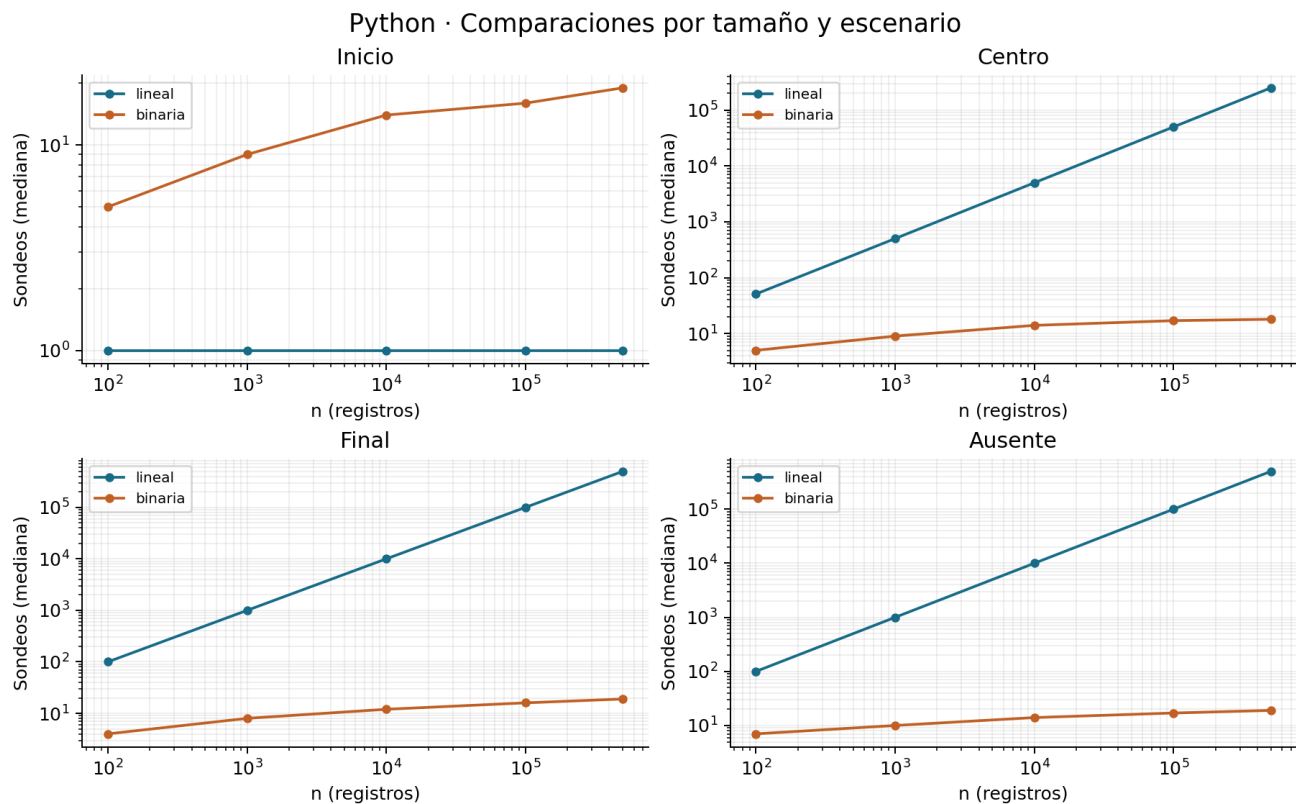


Figura 1. Mediana de sondeos por tamaño y escenario. Fuente: ejecuciones reales de este proyecto. Inicio/centro/final designan posiciones en la muestra original.

Escenario, $n = 100\,000$	Lineal	Binaria
Inicio	1	16
Centro	50 001	17
Final	100 000	16
Ausente	100 000	17

Tabla 1. Sondeos, iguales en ambos lenguajes. Las repeticiones no cambian este conteo determinista.

El inicio lineal conserva un sondeo para todo n . El centro requiere $\lfloor n/2 \rfloor + 1$, el final requiere n y la ausencia también n . Al aumentar de 100 a 500 000, final y ausente crecen por un factor de 5 000: se cumple la predicción lineal. La ausencia binaria pasa de 7 a 19 sondeos, consistente con el límite logarítmico.

Los casos exitosos de binaria no tienen que crecer monótonamente: cada muestra tiene claves y posiciones ordenadas diferentes. Lo que se contrasta es una cota, no una obligación de consumir exactamente $\log_2 n$ sondeos en cada consulta. Tampoco se espera que el centro original sea su mejor caso.

6.2. Lectura de la evidencia

El contador muestra crecimiento sin depender de la frecuencia del procesador. El reloj agrega información sobre constantes y entorno. Esta separación evita confundir escalabilidad con velocidad, distinción tratada por el repertorio de algoritmos del NIST (Black, 2020). La evidencia completa se conserva en `resultados_crudos.csv` y `resumen_estadistico.csv`; la tabla de esta página resume sondeos, no sustituye los estadísticos temporales.

6.3. Tiempos observados y costo de preparación

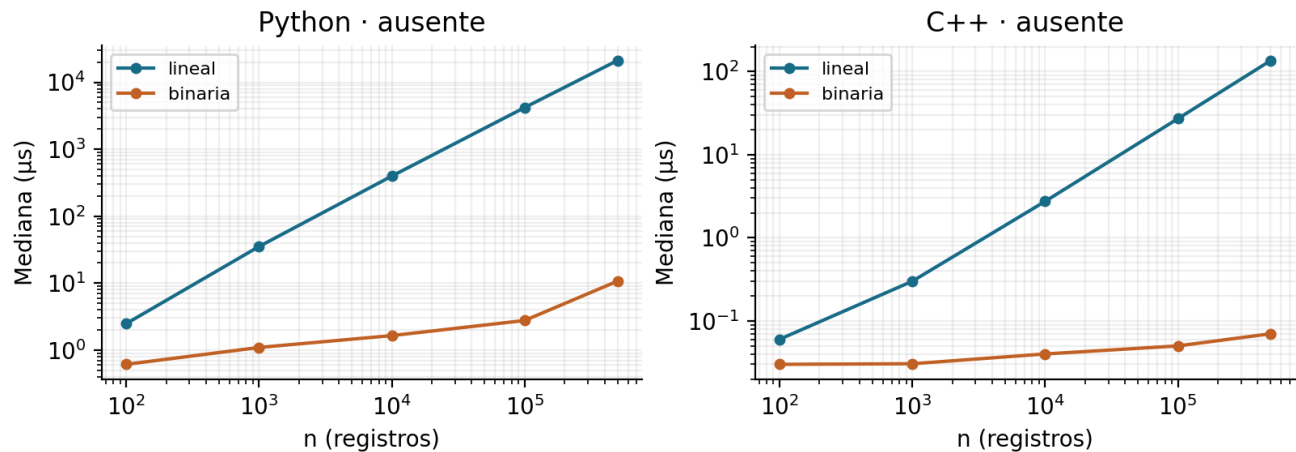


Figura 2. Tiempo mediano del caso ausente, por lenguaje, en microsegundos y ejes logarítmicos. Los cuatro escenarios de cada lenguaje se entregan en figuras separadas.

Lenguaje	Escenario, $n = 100\,000$	Lineal (μs)	Binaria (μs)
Python	Inicio	0.640	3.410
Python	Centro	2 075.443	3.270
Python	Final	4 185.722	2.969
Python	Ausente	4 183.473	2.760
C++	Inicio	0.030	0.050
C++	Centro	13.590	0.055
C++	Final	27.140	0.050
C++	Ausente	27.140	0.050

Tabla 2. Medianas de 30 ejecuciones, redondeadas a 0.001 μs . El CSV conserva mayor precisión, promedio, mínimo y máximo.

En ausencia con $n = 500\,000$, las medianas son 21.106 ms y 10.676 μs en Python, frente a 135.952 μs y 0.070 μs en C++ (lineal y binaria, respectivamente). Son observaciones del código instrumentado en Work; especialmente los intervalos pequeños dependen del costo del reloj, caché y planificación del sistema.

Una consulta frente a muchas. Para datos estables, $C_L(q) = qL$ y $C_B(q) = S + qB$, donde S es ordenar. Si $L > B$, binaria ofrece ventaja estricta cuando $q > S/(L - B)$. En ausencia con $n = 500\,000$, S mediano fue 73.373 ms en Python y 19.209 ms en C++. El primer entero favorable es $q = 4$ y $q = 142$, respectivamente.

Lenguaje	q	qL (ms)	$S + qB$ (ms)
Python	1	21.106	73.384
Python	1000	21 105.963	84.049
C++	1	0.136	19.209
C++	1000	135.952	19.279

Tabla 3. Costos *modelados* con medianas, para la misma consulta ausente y $n = 500\,000$; no son lotes adicionales cronometrados. Se excluyen carga y copia.

Para inicio lineal no hay amortización favorable en esta sesión porque $L \leq B$. La conclusión depende de la consulta y del costo de mantener la colección: afirmar que binaria siempre gana sería incorrecto.

7. Discusión de discrepancias y limitaciones

Los sondeos siguen la teoría, pero las medianas temporales no son funciones puras de n . El intérprete, la representación de enteros, la optimización del compilador, la localidad de acceso y la carga compartida alteran constantes. Se incluyó el contador y no se descontó el costo del reloj. Expresar un intervalo en nanosegundos no garantiza precisión física de un nanosegundo. No se fijó afinidad ni se aisló toda actividad del sistema.

Se usa una sola muestra por tamaño, 30 repeticiones de cada clave y calentamiento; por tanto no hay inferencia estadística sobre todas las transacciones o sobre consultas en frío. Las claves únicas elegidas condicionan deliberadamente los escenarios; los duplicados restantes se conservan y el tamaño 500 000 usa reemplazo. El centro no estima por sí solo el caso promedio. Los estadísticos son descriptivos y no se construyeron intervalos de confianza.

La amortización supone costos por consulta estables y excluye copia, carga y actualizaciones. Una tabla hash podría ofrecer búsqueda esperada constante con memoria y colisiones; un árbol equilibrado ofrece búsqueda y actualización logarítmicas. Elegir una estructura requiere considerar el trabajo completo, no solo una llamada (Skiena, 2020). Los benchmarks y pruebas automatizadas se ejecutaron en el entorno remoto Linux/GCC de Work. La demo fue reproducida además en la PC del estudiante con Windows 10, Python 3.14.5 y MinGW GCC 14.2.0 incluido con Code::Blocks; se comprobó búsqueda lineal, búsqueda binaria y el rechazo de la búsqueda binaria cuando los datos están desordenados.

8. Conclusiones

1. La búsqueda lineal verifica costo constante al inicio y crecimiento lineal en centro, final y ausencia. Binaria respeta el límite logarítmico y necesita orden previo. Los conteos concuerdan en ambos lenguajes.
2. Las cotas O , Ω y Θ describen funciones de costo, mientras los casos describen entradas o distribuciones. Los tiempos observados no reemplazan la derivación RAM.
3. Ordenar puede compensar con suficientes consultas, pero no siempre con una sola ni para objetivos al inicio. Los umbrales calculados pertenecen a esta sesión y sus supuestos.
4. Los datos con procedencia, crudos, versiones, pruebas y comandos permiten repetir el experimento. La demo se reprodujo localmente en Windows y el repositorio se publicó con commits reales separados en **baseline**, **analysis**, **benchmark** y **final**.

9. Declaración de uso de IA

Se utilizó ChatGPT para estructuración, apoyo de programación, revisión y documentación. Las implementaciones y los benchmarks se sometieron a pruebas y ejecución en Work; adicionalmente, las demos de Python y C++ fueron ejecutadas localmente en la PC del estudiante para verificar su funcionamiento y el manejo del requisito de ordenamiento. Los integrantes son responsables de revisar y comprender el trabajo antes de su sustentación.

10. Referencias

- Black, P. E. (2020). *DADS: The on-line dictionary of algorithms and data structures* (NISTIR 8318). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.IR.8318>
- Black, P. E. (2022). *Binary search*. En *Dictionary of Algorithms and Data Structures*. National Institute of Standards and Technology. <https://www.nist.gov/dads/HTML/binarySearch.html>
- Chen, D. (2015). *Online Retail* [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5BW33>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- ISO/IEC JTC1/SC22/WG21. (s. f.). *Class steady_clock [time.clock.steady]*. En *C++ working draft*. Recuperado el 10 de septiembre de 2026, de <https://eel.is/c++draft/time.clock.steady>
- Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3.^a ed.). Addison-Wesley Professional. <https://www.informit.com/store/art-of-computer-programming-volume-1-fundamental-algorithms-9780201896831>
- Levitin, A. (2012). *Introduction to the design and analysis of algorithms* (3.^a ed.). Pearson. <https://www.pearson.com/en-gb/subject-catalog/p/introduction-to-the-design-and-analysis-of-algorithms-international-edition/P200000004735>
- McGeoch, C. C. (2012). *A guide to experimental algorithmics*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511843747>
- Python Software Foundation. (s. f.). *time – Time access and conversions*. En *Python documentation*. Recuperado el 10 de septiembre de 2026, de <https://docs.python.org/3/library/time.html>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley Professional. <https://algs4.cs.princeton.edu/home/>
- Skiena, S. S. (2020). *The algorithm design manual* (3.^a ed.). Springer. <https://doi.org/10.1007/978-3-030-54256-6>
- Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio: Semana 2. Algoritmos y Estructuras de Datos, SIS210*

[Material de clase]. Universidad Nacional del Altiplano.